Lecture 38: Deep Learning (Part-3) — Practice Problem Solutions

Complete mathematical derivations, architectural proofs, and code implementations for all 15 practice problems.

Solution 1: Regression vs Multi-Class Classification Architecture

- **Regression**: The target $y \in \mathbb{R}$ is an unbounded continuous value (e.g. electrical power $420 - 495$ MW). The output layer requires 1 **neuron**. No non-linear activation is applied (linear output $z = Wx + b$) because applying ReLU would restrict predictions to non-negative numbers, and Sigmoid would restrict them to $[0, 1]$. The loss function is **Mean Squared Error** (`nn.MSELoss()`).
- **Multi-Class Classification**: The target $y \in \{0, 1, \ldots, C-1\}$ represents one of C mutually exclusive classes ($C = 7$ for date fruits). The output layer requires $C = 7$ **neurons**, each emitting an unnormalized score (logit) z_c. In PyTorch, we do NOT include `nn.Softmax()` in the model because `nn.CrossEntropyLoss()` applies `LogSoftmax` internally.

Solution 2: Mathematical Necessity of Feature Standardization

When features have vastly different scales ($x_1 \approx 500,000$ vs $x_2 \approx 0.7$), the gradient with respect to weight w_1 is proportional to x_1, while $\frac{\partial L}{\partial w_2}$ is proportional to x_2:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z} \cdot x_j$$

This creates an **extremely elongated, elliptical loss contour**. A single learning rate η causes gradient descent to oscillate violently along the steep axis (w_1) while making almost zero progress along the gentle axis (w_2). Standardization centers each feature at zero mean and unit variance ($z = \frac{x-\mu}{\sigma}$), transforming the elliptical contours into symmetrical hyperspherical bowls where gradient descent steps directly toward the global minimum.

Solution 3: Data Leakage in Feature Scaling

- **Data Leakage** occurs when information from the test dataset contaminates the training phase. If `fit_transform` is called on the entire dataset prior to splitting, the

scaler computes:

$$\mu_{\text{all}} = \frac{1}{N_{\text{all}}} \sum_{i=1}^{N_{\text{all}}} x_i, \quad \sigma_{\text{all}} = \sqrt{\frac{1}{N_{\text{all}}} \sum_{i=1}^{N_{\text{all}}} (x_i - \mu_{\text{all}})^2}$$

The training set is now scaled using statistics that contain information from the test set!

- **Correct Protocol**:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Compute mu and sigma fro
X_test_scaled = scaler.transform(X_test)       # Apply train mu and sigma
```

Solution 4: PyTorch Data Pipeline Mechanics

1. `TensorDataset` : Bundles feature and label tensors together index-by-index so that `dataset[i]` returns `(x_i, y_i)` .
2. `DataLoader` : Provides an iterable wrapper around `TensorDataset` that handles:
    - Mini-batching: Grouping individual samples into batches of size $B = 32$.
    - Shuffling: Permuting dataset indices every epoch when `shuffle=True` .
    - Multi-process data loading via `num_workers` .
3. **Why Mini-Batching?** Passing all samples at once (Batch Gradient Descent) requires holding massive intermediate activation tensors in RAM/VRAM, which crashes on large datasets. Mini-batching ($B = 32$) keeps memory usage constant and introduces mild stochastic gradient noise that helps gradient descent escape saddle points and shallow local minima.

Solution 5: Tensor Data Types (`torch.float32` vs `torch.long`)

- **Regression (`nn.MSELoss`)**: Computes squared differences between floats:

$$\text{Loss} = \frac{1}{B} \sum_{i=1}^{B} (\hat{y}_i - y_i)^2$$

Both predictions $\hat{y}$ and ground truth y must be continuous floats (`torch.float32`).

- **Multi-Class Classification (`nn.CrossEntropyLoss`)**: PyTorch implements negative log-likelihood by indexing directly into the predicted log-probability tensor using target class indices:

$$\text{Loss} = -\log P(y_i|\mathbf{x}_i) = -\text{log_softmax}(z)[i, y_i]$$

  Since y_i serves as an array lookup index ($0, 1, \ldots, C-1$), it must be an integer (`int64` / `torch.long`).
- **Runtime Error**: Passing `torch.float32` targets to `nn.CrossEntropyLoss` raises: `RuntimeError: Expected object of scalar type Long but got scalar type Float for argument #2 'target'` .

Solution 6: Manual Parameter Count Verification

For any fully-connected layer `nn.Linear(in_features, out_features)` :

$$\text{Parameters} = (\text{in_features} \times \text{out_features}) + \text{out_features (biases)}$$

1. Regression Network:

- Layer 1 (`Linear(4, 64)`): $4 \times 64 + 64 = 256 + 64 = 320$
- Layer 2 (`Linear(64, 32)`): $64 \times 32 + 32 = 2048 + 32 = 2080$
- Layer 3 (`Linear(32, 1)`): $32 \times 1 + 1 = 32 + 1 = 33$
- **Total**: $320 + 2080 + 33 = 2,433$ parameters.

2. Classification Network:

- Layer 1 (`Linear(34, 64)`): $34 \times 64 + 64 = 2176 + 64 = 2,240$
- Layer 2 (`Linear(64, 64)`): $64 \times 64 + 64 = 4096 + 64 = 4,160$
- Layer 3 (`Linear(64, 7)`): $64 \times 7 + 7 = 448 + 7 = 455$
- **Total**: $2,240 + 4,160 + 455 = 6,855$ parameters.

Solution 7: The Softmax & `nn.CrossEntropyLoss` Trap

PyTorch's `nn.CrossEntropyLoss()` applies `nn.LogSoftmax()` followed by `nn.NLLLoss()` internally:

$$\text{Loss}(z, y) = -\log\left(\frac{e^{z_y}}{\sum_j e^{z_j}}\right) = -z_y + \log\sum_j e^{z_j}$$

To prevent numerical overflow (when $e^{z_j} \to \infty$) or underflow, PyTorch evaluates this using the **Log-Sum-Exp trick**:

$$\log \sum_j e^{z_j} = c + \log \sum_j e^{z_j - c}, \quad \text{where } c = \max_j(z_j)$$

If you explicitly add `nn.Softmax()` to your network architecture, the output values become probabilities $p \in (0, 1)$. Passing these probabilities to `nn.CrossEntropyLoss` causes it to treat probabilities as raw logits and apply `LogSoftmax` a second time, severely compressing gradients and destroying optimization!

Solution 8: The Four Pillars of the PyTorch Optimization Step

1. **`optimizer.zero_grad()`** : Clears previously computed gradients (`p.grad.zero_()`). In PyTorch, gradients **accumulate** (`p.grad += dL/dp`) by default to facilitate gradient accumulation across mini-batches.
2. **`outputs = model(inputs)`** : Executes the forward pass, building the dynamic computational graph.
3. **`loss.backward()`** : Traverses the computational graph backward using reverse-mode autodiff to compute $\frac{\partial \text{Loss}}{\partial w}$ for all parameters.
4. **`optimizer.step()`** : Updates model parameters using the calculated gradients:

$$w \leftarrow w - \eta \cdot \nabla_w L$$

- **What happens if `zero_grad()` is omitted?** Gradients from previous batches accumulate endlessly. On batch k, the gradient update would be $\sum_{j=1}^{k} \nabla L_j$. The gradient magnitudes explode, causing weights to oscillate wildly and weights/losses to diverge to `NaN`.

Solution 9: `model.state_dict()` vs Full Model Serialization

- **`model.state_dict()`** is a standard Python dictionary mapping each parameter string name to its torch tensor (e.g. `'model.0.weight' -> tensor(...)`).
- **Why avoid `torch.save(model, path)`?** Saving the model object serializes the class structure using `pickle` . If the class definition is moved, renamed, refactored, or loaded in a environment with different file paths or PyTorch versions, `pickle` throws `AttributeError: Can't get attribute 'ANN'` .
- **Best Practice**: Saving `state_dict` serializes pure numerical weight arrays. It is lightweight, secure, and enables loading weights into any refactored script or

deployment microservice:

```
torch.save(model.state_dict(), "best_model.pt")
# Restore
model = ANN()
model.load_state_dict(torch.load("best_model.pt"))
```

Solution 10: Model Checkpointing and Overfitting Mitigation

- **Rationale**: As training iterations advance, training loss monotonically decreases. However, validation loss begins increasing after reaching a minimum once the network starts memorizing training noise. Saving at the final epoch saves an overfitted model. Checkpointing ensures we preserve parameters from the epoch of optimal generalization.
- **Why track validation loss?** Validation data is held out from gradient updates; its performance objectively measures unseen generalization.
- **Why `float('inf')`?** Initializing `best_val_loss = float('inf')` guarantees that the validation loss from epoch 1 will always be smaller, correctly initializing the checkpointing threshold on the very first comparison.

Solution 11: Evaluation Guardrails (`model.eval()` and `torch.no_grad()`)

- `model.eval()` : Modifies layer internal execution modes. It tells layers like `Dropout` to deactivate (dropout probability $p = 0$) and `BatchNorm` to use fixed running mean and variance rather than batch statistics. **It does NOT disable gradient calculation.**
- `with torch.no_grad():` : A context manager that disables autograd engine tracking. It prevents PyTorch from building the directed computational graph and caching intermediate activation tensors, reducing memory consumption by up to 70% and speeding up forward inference. **It does NOT alter layer behaviors like dropout.**
- Therefore, **both** are required during model evaluation:

```
model.eval()
with torch.no_grad():
    predictions = model(X_test_tensor)
```

Solution 12: Dynamic Batch Size Handling in Accuracy Calculation

The test dataset has $N_{\text{test}} = 180$ samples. With `batch_size = 32` :

- Batch 1: 32 samples

- Batch 2: 32 samples
- Batch 3: 32 samples
- Batch 4: 32 samples
- Batch 5: 32 samples
- Batch 6 (Partial remainder batch): $180 - (5 \times 32) = 20$ samples. If you write `total += 32` inside the loop, after 6 batches `total` evaluates to $6 \times 32 = 192$. Then `accuracy = correct / 192 * 100`, which erroneously deflates your true accuracy by dividing by 192 instead of 180! `total += yb.size(0)` dynamically inspects the tensor's 0-th dimension, returning 32 for the first 5 batches and 20 for the final batch, correctly summing to 180.

Solution 13: Regression Metric Interpretation

1. **RMSE Calculation**:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{17.81} \approx 4.22 \text{ MW}$$

Physical Meaning: On unseen test operating conditions, the neural network's electrical power prediction deviates from the true power plant output by an average of approximately ± 4.22 Megawatts. Given the plant output spans 420 MW to 495 MW (a 75 MW range), this represents a relative error of only:

$$\text{Relative Error} = \frac{4.22}{495 - 420} \approx 5.6\% \text{ of span (or } \approx 0.9\% \text{ of mean o}$$

2. $R^2 = 0.937$ **Interpretation**: The artificial neural network explains 93.7% **of the variance** in net hourly electrical energy output using the 4 ambient sensor measurements. Only 6.3% of the variance is unexplained (residual error).

Solution 14: Argmax Equivalence over Logits vs Probabilities

For a given input vector, the network produces raw logits $\mathbf{z} = [z_1, z_2, \ldots, z_C]$. The Softmax probability for class i is:

$$P(y = i|\mathbf{x}) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Since the denominator $D = \sum_{j=1}^{C} e^{z_j}$ is strictly positive and constant across all classes $i \in \{1, \ldots, C\}$, and since the exponential function $f(u) = e^u$ is strictly monotonically increasing:

$$z_a > z_b \iff e^{z_a} > e^{z_b} \iff \frac{e^{z_a}}{D} > \frac{e^{z_b}}{D} \iff P(y=a|\mathbf{x}) > P(y=b|\mathbf{x})$$

Therefore:

$$\arg\max_i P(y=i|\mathbf{x}) \equiv \arg\max_i z_i$$

Computing `torch.max(outputs, 1)` directly on raw logits guarantees the exact same predicted class as computing it on Softmax probabilities, saving unnecessary exponential computations during inference.

Solution 15: Dimensionality Reduction Challenge (PCA + ANN)

```
from sklearn.decomposition import PCA
import torch
import torch.nn as nn

# 1. Fit PCA on training features to retain 95% cumulative variance
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

k = X_train_pca.shape[1]
print(f"Features reduced from 34 to {k} principal components!")

# 2. Convert to PyTorch Tensors
X_train_tensor = torch.tensor(X_train_pca, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test_pca, dtype=torch.float32)

# 3. Revised Model Architecture
class PCABasedANN(nn.Module):
    def __init__(self, in_features, num_classes=7):
        super(PCABasedANN, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(in_features, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, num_classes)
        )
    def forward(self, x):
        return self.model(x)

model = PCABasedANN(in_features=k, num_classes=7)
```

This reduces parameter count in Layer 1 from $34 \times 64 = 2,176$ to $k \times 64$, speeds up training, mitigates collinearity among morphological features, and preserves high classification accuracy.